# SOFTWARE REQUIREMENTS SPECIFICATION

*Project Blueprint & Implementation Guide*

# HOTEL MANAGEMENT SYSTEM (HMS)

**A Web-Based Hotel Operations Platform (Next.js / Node.js / PostgreSQL)**

Document Type: SRS + System Design + Implementation Blueprint
Version: 3.0 (Web Stack Revision)
Date: July 2026

# 1. Introduction & Project Overview

## 1.1 Overview

The Hotel Management System (HMS) is a centralized web platform that automates the full operational lifecycle of a hotel — room inventory, reservations, check-in/check-out, billing, housekeeping, restaurant service, staff management, and reporting — replacing manual registers and disconnected spreadsheets with one live, shared source of truth used by every department, accessible from any browser on any front-desk, housekeeping, or management device.

When a guest arrives at a hotel that runs on a system like this, the receptionist finds the booking in seconds, assigns a room, and the room's status instantly updates — in real time, over the network — for housekeeping and restaurant staff on their own devices. At checkout, all charges — room, food, services — are combined automatically into a single accurate invoice. HMS delivers exactly this experience for a single-property hotel, with every role signing in through the same web application at its own URL.

## 1.2 Problem Statement

Small and mid-sized hotels commonly operate on manual or semi-digital processes, which creates a consistent set of operational failures:

| Problem | Impact |
|---|---|
| Paper-based or spreadsheet booking records | Double bookings, lost or inconsistent records |
| No real-time room status | Staff must physically check or call to see whether a room is clean or occupied |
| Disconnected billing across departments | Manual, error-prone tallying of room, food, and service charges at checkout |
| No centralized reporting | Management has no live view of occupancy, revenue, or staff performance |
| Device-bound tools (single desktop PC) | Only whoever is physically at that PC can see live data — a real bottleneck once multiple departments and terminals are involved |
| No remote visibility for management | Owners/managers cannot check occupancy or revenue unless they are on-site |

## 1.3 Proposed Solution

HMS solves this by giving every role — Admin, Receptionist, Housekeeping, Restaurant Staff, Manager — one shared, role-restricted web dashboard reading from the same live database, reachable from any browser on the hotel's network (or, if the owner chooses, over the internet). Core mechanisms include:

- A real-time room-status engine (Available, Occupied, Reserved, Cleaning, Maintenance) pushed to every connected dashboard over WebSockets the instant it changes.
- A booking engine that locks a room the moment it is reserved, structurally preventing double bookings at the database level.
- An automated billing engine that aggregates room, restaurant, and service charges into one itemized invoice, exportable as a PDF.
- A reporting layer with live charts that gives management instant visibility into occupancy and revenue without manual compilation, from any device.
- A guest-profile system that recognizes returning guests and preserves their stay history and preferences.
- Google / Microsoft single sign-on alongside standard email-and-password login, matching the login screen already designed for the product.

# 2. Project Scope & Objectives

## 2.1 In Scope (Version 1.0)

- Room inventory, room-type categorization, and pricing management.
- Full reservation lifecycle: booking (walk-in, phone, or staff-entered online request) → confirmation → check-in → stay → check-out.
- Role-based accounts for Admin, Manager, Receptionist, Housekeeping, and Restaurant staff, with guest profiles managed separately, all authenticated through the same web app.
- Real-time room availability checking with double-booking prevention, synced live across every open dashboard.
- Billing, itemized invoicing, and payment recording (Cash, Card, Bank Transfer, Wallet).
- In-house restaurant / room-service ordering linked automatically to the guest's room folio.
- Housekeeping task tracking per room, from post-checkout cleaning through inspection, viewable on any tablet or PC.
- Staff records, shift tracking, and an audit log of system actions.
- Reports and dashboards: occupancy, revenue, room-type performance, guest demographics, rendered as live, interactive charts.
- Post-stay feedback and multi-category ratings (room, service, food, overall).
- In-app, role-targeted, real-time notifications (booking created, room ready, order placed, negative feedback, etc.).
- Google / Microsoft OAuth login in addition to email + password, as shown in the approved login screen design.

### 2.2 Out of Scope (Version 1.0 — Reserved for Future Phases)

- Customer-facing self-service public booking website and native mobile app (staff enter online/phone bookings directly in v1; the internal web app is not the same thing as a public booking site).
- Third-party OTA integration (Booking.com, Expedia, Agoda).
- Real payment-gateway integration (Stripe/JazzCash/Easypaisa) — payments are recorded, not processed online, in v1.
- Multi-property / hotel-chain management (single-tenant, single-hotel database in v1).
- IoT integration (smart locks, smart thermostats) and self-service kiosks.
- Full accounting/payroll suite — HMS manages staff records, not salary processing or tax filing.
- Native offline mode — v1 assumes a working internet/LAN connection; offline-first PWA caching is a Phase 2 candidate.

### 2.3 Objectives

- Eliminate manual, paper-based record-keeping across all departments.
- Reduce guest check-in/check-out time from roughly 10 minutes to under 3 minutes.
- Achieve zero double-bookings through database-enforced availability locking.
- Give every department real-time visibility into room status, over the network, without cross-department phone calls.
- Provide management with on-demand, exportable reports on occupancy, revenue, and staff performance, viewable from any browser, including off-site.
- Deliver an interface simple enough that new front-desk staff are productive after about one hour of training.
- Build a codebase where frontend and backend are cleanly separated by a versioned REST/WebSocket API, so a future mobile app or public booking portal can reuse the same backend without a rewrite.

## 3. Requirements Analysis

### 3.1 Functional Requirements

Each requirement is uniquely numbered for traceability into test cases and the formal SRS. Requirements are grouped by module. These requirements are unchanged from the original blueprint — they describe hotel operations, not implementation technology — and apply identically to the web-based architecture.

### 3.1.1 Authentication & Access Control

| ID | Requirement |
|---|---|
| FR-001 | The system shall allow users to register with email, password, name, phone, and role. |
| FR-002 | The system shall allow login via email and password, with credentials verified against a bcrypt-hashed password over HTTPS. |
| FR-003 | The system shall additionally support Google and Microsoft OAuth 2.0 login, mapping the OAuth account to an existing staff record. |
| FR-004 | The system shall issue a signed JWT access token (and refresh token) on successful login, scoped to the user's role. |
| FR-005 | The system shall lock an account after 5 consecutive failed login attempts and require Admin unlock or a timed cooldown. |
| FR-006 | The system shall support a 'Forgot Password' flow that emails a time-limited reset link. |

### 3.1.2 Admin Management

| ID | Requirement |
|---|---|
| FR-010 | Admin can create, read, update, and delete (CRUD) staff accounts. |
| FR-011 | Admin can assign and revoke roles and permissions. |
| FR-012 | Admin can configure hotel-wide settings: name, logo, tax rate, currency, default check-in/check-out times. |
| FR-013 | Admin can view a full audit log of system actions, filterable by user, module, and date. |

### 3.1.3 Room Management

| ID | Requirement |
|---|---|
| FR-020 | The system shall support configurable room categories (e.g., Single, Double, Deluxe, Suite, Family). |
| FR-021 | Each room record shall store: number, floor, type, price, status, and amenities. |
| FR-022 | Room status changes made by any role shall be broadcast in real time to every connected dashboard via WebSocket, without a page refresh. |
| FR-023 | Admin/Manager can bulk-update pricing by room type or season. |

### 3.1.4 Booking & Reservation Management

| ID | Requirement |
|---|---|
| FR-030 | Staff can search rooms by date range, guest count, and room type. |
| FR-031 | The system shall display matching available rooms with price, amenities, and images. |
| FR-032 | Receptionist can create a booking for a walk-in, phone, or staff-entered online guest. |
| FR-033 | The system shall calculate nights, subtotal, tax, and total automatically as the booking is built. |
| FR-034 | The system shall allow booking modification and cancellation, subject to role permissions. |

### 3.1.5 Room Availability

| ID | Requirement |
|---|---|
| FR-040 | The system shall check availability in real time against check-in date, check-out date, room type, and guest count. |
| FR-041 | The system shall prevent double-booking through a database-level constraint (PostgreSQL exclusion constraint on room + date range) that rejects overlapping reservations for the same room. |

### 3.1.6 Check-In System

| ID | Requirement |
|---|---|
| FR-050 | Receptionist can search a booking by reference number or guest name. |

| ID | Requirement |
|---|---|
| FR-051 | The system shall display full booking details and the assigned room. |
| FR-052 | On check-in, the system shall atomically set booking status to Checked-In and room status to Occupied, and push both updates live to Housekeeping and Restaurant dashboards. |

### 3.1.7 Check-Out & Room Allocation

| ID | Requirement |
|---|---|
| FR-060 | Receptionist can initiate checkout from the active-guest list. |
| FR-061 | The system shall compile all charges — room nights, restaurant orders, and services — into a single folio. |
| FR-062 | On checkout completion, the system shall set room status to Cleaning and automatically create a housekeeping task, visible instantly on the Housekeeping dashboard. |

### 3.1.8 Payment & Billing

| ID | Requirement |
|---|---|
| FR-070 | The system shall support Cash, Card, Bank Transfer, and Wallet as recorded payment methods. |
| FR-071 | The system shall record partial payments and track the remaining balance. |
| FR-072 | The system shall support applying a discount, with an audit-logged reason. |

### 3.1.9 Invoice Generation

| ID | Requirement |
|---|---|
| FR-080 | The system shall auto-generate an invoice on completed payment. |
| FR-081 | The invoice shall include hotel details, guest details, itemized charges, tax, total, and payment method. |
| FR-082 | The system shall render the invoice as a downloadable PDF, generated server-side. |

### 3.1.10 Staff Management

| ID | Requirement |
|---|---|
| FR-100 | The system shall store staff records: name, role, department, contact, hire date, and status. |
| FR-101 | Admin can assign staff to shifts. |

### 3.1.11 Food / Restaurant Services

| ID | Requirement |
|---|---|
| FR-110 | Restaurant staff can view the current list of in-house guests. |
| FR-111 | Staff can create a food/room-service order against a guest's room number, with items, quantities, and special instructions. |
| FR-112 | New and updated orders shall appear live on the Restaurant dashboard's order queue via WebSocket, without polling. |

### 3.1.12 Housekeeping Management

| ID | Requirement |
|---|---|
| FR-120 | The system shall automatically queue a room for cleaning once checkout is completed. |
| FR-121 | A housekeeping supervisor can assign tasks to specific staff. |
| FR-122 | Staff can mark a task complete from any device (tablet or desktop browser), instantly flipping the room back to Available for Reception. |

### 3.1.13 Reports & Statistics

| ID | Requirement |
|---|---|
| FR-130 | The dashboard shall show today's occupancy rate, revenue, check-ins, and check-outs, as live-updating summary cards. |

| ID | Requirement |
|---|---|
| FR-131 | The system shall produce occupancy and revenue reports by day, week, month, and room type, rendered as interactive charts. |
| FR-132 | Reports shall be exportable as PDF and CSV. |

### 3.1.14 Feedback & Review System

| ID | Requirement |
|---|---|
| FR-140 | Guests can submit post-stay feedback with multi-category ratings (room, service, food, overall). |
| FR-141 | Negative feedback shall trigger an in-app notification to the Manager role. |

### 3.1.15 Notifications

| ID | Requirement |
|---|---|
| FR-150 | The system shall provide an in-app notification bell with an unread count, updated live via WebSocket. |
| FR-151 | Trigger events shall include: new booking, check-in, check-out, room ready, food order, and negative feedback. |

## 3.2 Non-Functional Requirements

| ID | Category | Requirement |
|---|---|---|
| NFR-001 | Performance | The main dashboard shall load within 2 seconds on a broadband/LAN connection. |
| NFR-002 | Response Time | A room-availability check shall complete in under 500 milliseconds server-side. |
| NFR-003 | Real-Time | Room-status and notification updates shall reach all connected clients within 1 second via WebSocket. |
| NFR-004 | Scalability | The backend shall be stateless (session state in the database/JWT, not in-process) so it can run multiple instances behind a load balancer if the hotel later needs it. |
| NFR-005 | Security | All traffic shall be served over HTTPS/WSS; passwords hashed with bcrypt; JWTs short-lived with refresh rotation. |
| NFR-006 | Availability | Target 99.5% uptime for the hosted application, backed by managed database backups. |
| NFR-007 | Browser Support | The web app shall run correctly on the latest two versions of Chrome, Edge, Firefox, and Safari. |
| NFR-008 | Responsiveness | Core screens (Dashboard, Room Grid, Booking, Checkout) shall be usable on both desktop and tablet viewport widths. |
| NFR-009 | Auditability | Every create/update/delete on a booking, payment, or room status shall be recorded in the audit log with actor, timestamp, and before/after state. |

## 3.3 User Roles & Permissions

| Feature | Admin | Manager | | | |
|---|---|---|---|---|---|
| System settings | Full | View | — | — | — |
| Staff management | Full | View | — | — | — |
| Room management | Full | Full | View | — | — |
| Bookings & check-in/out | Full | Full | Full | — | — |
| Billing & payments | Full | Full | Full | — | — |
| Housekeeping tasks | Full | Full | View | Full (own tasks) | — |
| Restaurant orders | Full | Full | View | — | Full |
| Reports & analytics | Full | Full | View (own shift) | — | — |
| Audit logs | Full | View | — | — | — |

## 3.4 System Constraints

- Version 1.0 targets a single hotel property — no multi-branch synchronization.
- The web application requires a working internet or local-network connection; there is no offline mode in v1.
- Payments are recorded by staff, not processed through a live payment gateway, in v1.
- A single PostgreSQL database instance is used for v1 — no multi-region replication.
- Typical academic/capstone timeline assumption: 12 weeks, buildable solo or in a small team.

### 3.5 Assumptions

- The hotel has a fixed, known room inventory (roughly 20–500 rooms) organized into a small number of room types.
- Staff need only a modern web browser and basic computer literacy — no client software installation is required.
- Default check-in time is 14:00 and check-out time is 11:00, both configurable by Admin.
- Tax rate and currency are configurable per hotel, not hardcoded.
- The hotel provides reasonably stable Wi-Fi/LAN coverage across front desk, housekeeping, and restaurant areas.

## 4. System Architecture & Technology Stack

### 4.1 Architectural Decision

Two directions were considered for this project: a self-contained desktop application with an embedded database, and a networked 3-tier web architecture (browser client + REST/WebSocket API + database server). Because the UI is designed as a multi-device, multi-department dashboard — reception, housekeeping, and restaurant all need to see the same live room status from different physical terminals at the same time — the web approach is the better fit here. A single-machine desktop build would require its own networking layer to achieve the same live cross-department visibility, effectively rebuilding a web architecture from scratch.

The recommended architecture is a classic 3-tier web application: a browser-based frontend (Next.js/React), a backend API layer that owns all business logic and enforces booking/availability rules, and a managed PostgreSQL database. A WebSocket channel runs alongside the REST API purely for live push updates (room status, notifications, new orders) so dashboards never need manual refreshing. Because the frontend and backend are separate processes talking over a versioned API from day one, adding a mobile app or public booking portal later is an additive change, not a rewrite.

### 4.2 Recommended Technology Stack

| Layer | Technology | Why |
|---|---|---|
| Frontend | Next.js 14+ (App Router) with TypeScript | Server components for fast dashboard loads, file-based routing across role-specific screens, strong typing across a large multi-role app |
| Styling / UI Kit | Tailwind CSS + shadcn/ui | Matches the dark, card-based, professional dashboard aesthetic already designed, without a heavy custom CSS layer |
| Charts | Recharts | Drives the Revenue Trend, Occupancy donut, and Bookings Overview charts on the Reports screen |
| Client State / Data Fetching | TanStack Query (React Query) | Caches and auto-refetches server data (bookings, rooms) alongside live WebSocket pushes |
| Real-Time Layer | Socket.io (client + server) | Pushes room-status changes, new bookings, and notifications to every open dashboard instantly |
| Backend API | Node.js + Express (or NestJS for stricter module structure) | REST endpoints for bookings, billing, staff, reports; hosts the Socket.io server alongside it |
| ORM | Prisma | Type-safe queries mapped directly onto the ER design in Section 6; migrations tracked in source control |

| Layer | Technology | Why |
|---|---|---|
| Database | PostgreSQL | Enforces non-overlapping booking dates via exclusion constraints; handles concurrent multi-terminal writes far better than an embedded file database |
| Authentication | JWT (access + refresh tokens) + bcrypt, with Google/Microsoft OAuth via Passport.js or NextAuth | Matches the login screen's email/password + 'continue with Google/Microsoft' design |
| PDF Generation | PDFKit or Puppeteer (headless Chrome → PDF) | Server-side rendering of invoices and exported reports |
| Caching (optional) | Redis | Caches dashboard summary stats so repeated loads don't hit Postgres every time |
| File Storage | Cloudinary or S3-compatible bucket | Staff/guest profile photos, room images |
| Hosting — Frontend | Vercel | Zero-config Next.js deployment, preview URLs per branch |
| Hosting — Backend + DB | Railway or Render (managed PostgreSQL) | Low-cost, capstone/small-hotel friendly managed hosting with automatic backups |

### 4.3 API Structure

Unlike the desktop-first version — where the API was deferred to a hypothetical 'Phase 2' — the API is the core of the system from day one, since the browser frontend can only reach the database through it. Representative endpoints:

| Resource | Example Endpoints |
|---|---|
| Auth | POST /api/auth/login, POST /api/auth/register, POST /api/auth/oauth/google, POST /api/auth/refresh |
| Rooms | GET /api/rooms, GET /api/rooms/availability, PATCH /api/rooms/:id/status |
| Bookings | POST /api/bookings, GET /api/bookings/:id, PATCH /api/bookings/:id, POST /api/bookings/:id/checkin, POST /api/bookings/:id/checkout |
| Billing | GET /api/bookings/:id/folio, POST /api/payments, GET /api/invoices/:id/pdf |
| Housekeeping | GET /api/housekeeping/tasks, PATCH /api/housekeeping/tasks/:id |
| Restaurant | POST /api/orders, PATCH /api/orders/:id/status |
| Reports | GET /api/reports/occupancy, GET /api/reports/revenue, GET /api/reports/export |
| Realtime (WebSocket events) | room:status_changed, booking:created, order:created, notification:new |

### 4.4 Deployment Strategy

- Frontend (Next.js) deploys to Vercel directly from the Git repository; every push to main triggers a production build, every pull request gets a preview URL.
- Backend (Node.js/Express) and PostgreSQL deploy to Railway or Render; environment variables (DB connection string, JWT secret, OAuth client IDs) are stored in the platform's secret manager, never committed to source control.
- GitHub Actions runs lint, type-check, and automated tests on every pull request before merge.
- Database migrations run via `prisma migrate deploy` as a release step, so schema and code stay in lockstep.
- Daily automated PostgreSQL backups are enabled through the hosting provider's managed backup feature.

## 5. UI/UX Design

### 5.1 Design Principles

- Clarity over decoration — front-desk staff must find information in seconds.
- Consistent color coding for room status: green = Available, red = Occupied, blue = Reserved, yellow = Cleaning, grey = Maintenance.
- A collapsible sidebar drives navigation, matching the layout convention of professional PMS/SaaS dashboards.

- Dark-themed UI by default, reducing eye strain during long front-desk shifts, with the hotel's own logo and name in the sidebar/header.
- Fully responsive layout so the same dashboard works on a front-desk monitor, a housekeeping tablet, or a manager's phone browser.

## 5.2 Screen Flow

Login Screen (email/password or Google/Microsoft OAuth)

- → Admin Dashboard → Staff Management, System Settings, Audit Logs, All Reports
- → Receptionist Dashboard → New Booking, Check-In, Check-Out, Live Room Status Grid, Guest Search, Billing
- → Housekeeping Dashboard → Task List, Mark Complete
- → Restaurant Dashboard → New Order, Order Queue, Mark Served
- → Reports & Analytics → Revenue Trend, Occupancy Rate, Bookings Overview, Staff Performance, Guest Satisfaction Trend, Export PDF/CSV

## 5.3 Core Screens (as approved in the current design)

### Login Screen

Split layout: left panel shows hotel branding, tagline ('Smart Hotel Management Simplified'), and three feature highlights (All-in-One Solution, Secure & Reliable, Real-time Analytics) over a hotel photo. Right panel holds the email/password form, a 'Remember me' checkbox, 'Forgot password?' link, and 'Continue with Google / Microsoft' buttons, plus a 'Contact Administrator' link for account requests instead of open self-registration.

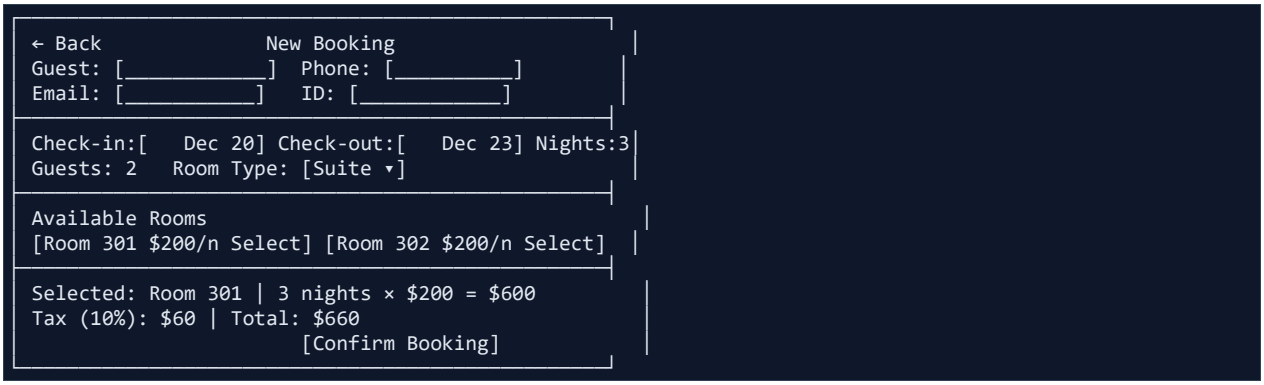
### Main Dashboard

Top summary cards: Today's Check-ins, Today's Check-outs, Occupancy Rate, Total Revenue, Available Rooms, Reserved Rooms — each with a live vs-yesterday delta. Quick-action buttons for New Booking, Check-In, Check-Out, and Generate Invoice sit directly beneath. Below that: a Recent Bookings table, a color-coded Live Room Status grid (Available/Occupied/Cleaning/Maintenance/Reserved), a Revenue Trend line chart, an Occupancy donut chart, and a live Notifications feed.

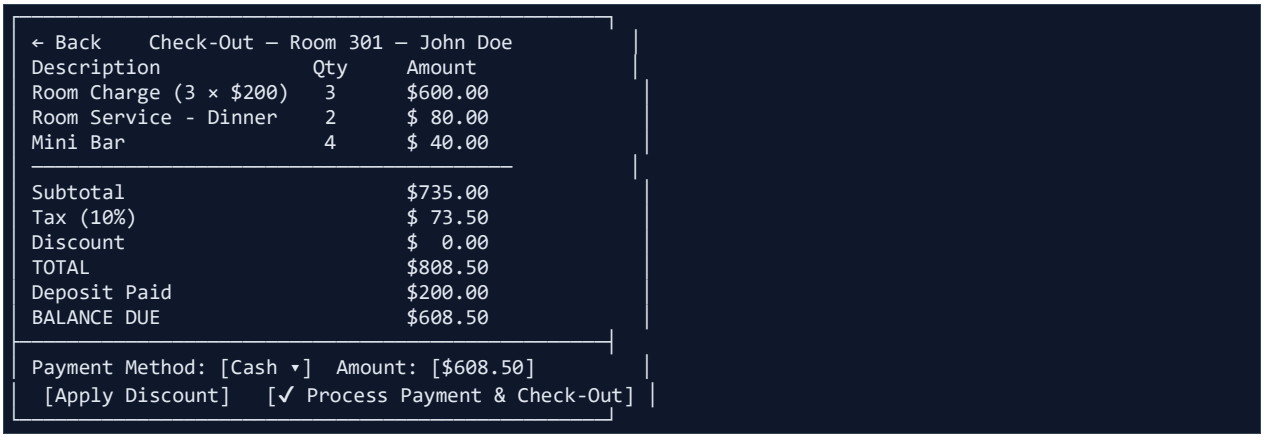
### Reports & Analytics

Date-range and department/room-type filters at the top, with Export PDF, Export CSV, and Print Report actions. Summary cards for Total Revenue, Occupancy Rate, Total Bookings, Average Length of Stay, and Guest Satisfaction. Below: Revenue Trend, Occupancy Rate donut, Bookings Overview bar chart, Restaurant Revenue, Staff Performance, and Revenue-by-Department donut, followed by detailed Revenue, Occupancy, Staff, and Restaurant report tables, and a Guest Satisfaction Trend line chart.

### Booking Screen



### Check-Out / Billing Screen




## 5.4 UI Component Standards

- Data tables with sorting, filtering, and pagination (client-side for small sets, server-side paginated for large ones).
- Form inputs with inline validation messages (React Hook Form + Zod schema validation, shared between client and server).
- Color-coded status badges for rooms, bookings, tasks, and orders.
- Modal confirmation dialogs for destructive or high-impact actions (delete, checkout, logout).
- Toast notifications for success/error feedback.
- A global quick-search bar across guests, rooms, and bookings.
- A persistent notification bell with an unread badge, fed live by the WebSocket channel.

# 6. Database Design

## 6.1 Entity-Relationship Overview

The database is built around a central bookings table linking a guest to a room for a specific date range. Payments, invoices, food orders, and feedback all trace back to a booking; housekeeping and maintenance records trace back to a room; users (staff) are kept separate from guests, since staff authenticate into the system while guests are the subjects being managed. All primary keys use UUIDs, and the schema is defined and versioned through Prisma migrations rather than hand-written SQL, so the schema in source control is always the schema running in production.

## 6.2 Key Relationships Summary

- roles (1) → users (many); users (1) → staff (1, optional — only hotel employees).
- guests exist independently of users, since guests do not log in.
- room_types (1) → rooms (many); rooms (1) → bookings (many, non-overlapping dates enforced by a PostgreSQL exclusion constraint).
- guests (1) → bookings (many); bookings (1) → payments (many), (1) → invoices (1), (1) → food_orders (many), (0 or 1) → feedback.
- invoices (1) → invoice_items (many); food_orders (1) → order_items (many).
- rooms (1) → housekeeping_tasks (many) and (1) → room_maintenance (many).
- users (1) → notifications (many) and (1) → audit_logs (many).

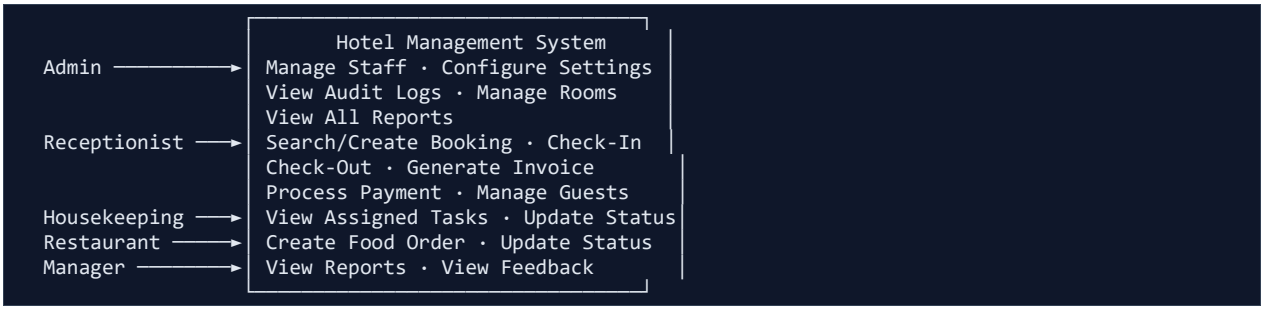
# 7. SRS Document Structure & UML Guide

Structure the formal SRS submission following the IEEE 830 / 29148 convention. This blueprint's content maps directly onto that structure as follows:

## 7.1 Use Case Descriptions

Document each major feature using: Actor(s), Preconditions, Main Flow, Alternate Flow, Postconditions. Repeat this structure for: Register/Login, Check-In Guest, Check-Out Guest, Generate Invoice, Assign Housekeeping Task, Create Food Order, Submit Feedback, and View Reports.

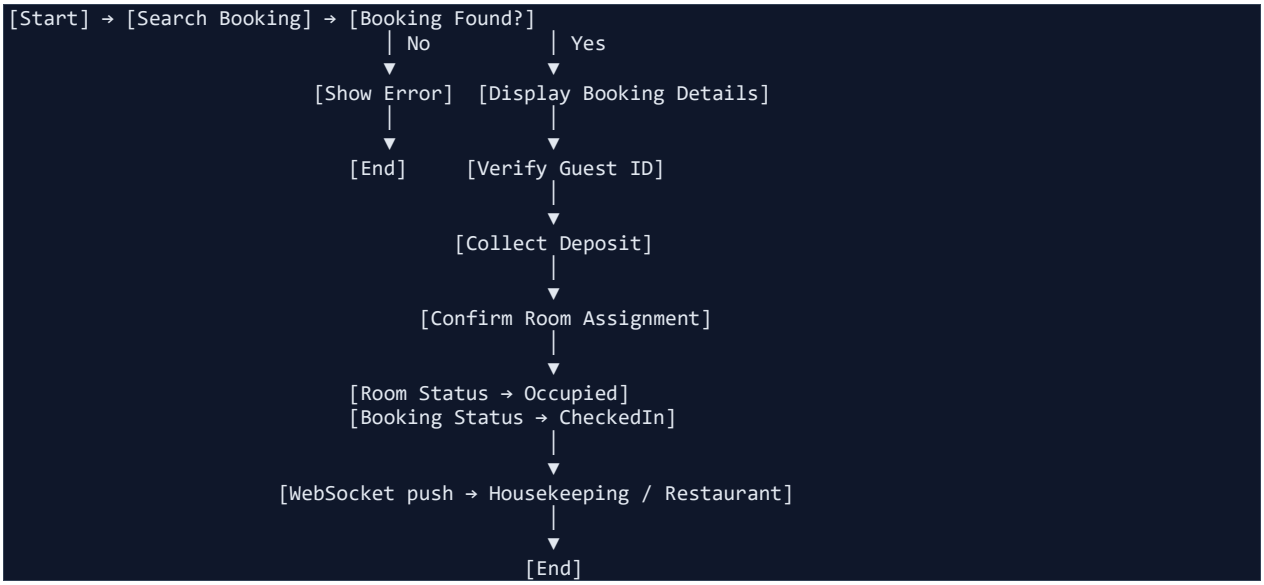
## 7.2 Use Case Diagram



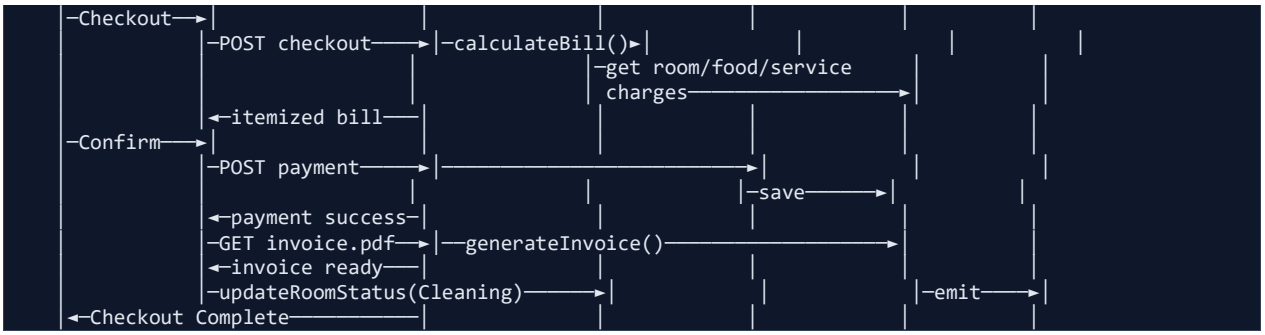

## 7.3 Class Diagram Guidance

Model the core entities from Section 6 as classes: Guest, Room, RoomType, Booking, Payment, Invoice, User, Staff, HousekeepingTask, FoodOrder. Each class carries the attributes from its matching table, plus behavior methods (e.g., Booking.calculateTotal(), Room.checkAvailability(dateRange)) — implemented as Prisma-model-backed service methods on the backend rather than in-process desktop classes. Represent Booking↔Guest and Booking↔Room as associations, and Invoice↔InvoiceItem as a composition.

## 7.4 Activity Diagram — Check-In Flow




## 7.5 Sequence Diagram — Checkout & Billing Flow




## 7.6 ER Diagram

Use the entity-relationship diagram in Section 6.1 as the basis for the formal ERD; render it in a tool such as draw.io, Lucidchart, or StarUML with explicit cardinality notation (1:1, 1:M) on each relationship line. The Prisma schema (schema.prisma) can also auto-generate a first-pass ERD via prisma-erd-generator.

# 8. Development Plan

## 8.1 Development Phases

| Phase | Duration | Deliverables |
|---|---|---|
| 1. Foundation | Week 1–2 | Repo setup (frontend + backend), Prisma schema + seed data, auth (JWT + OAuth), base Next.js layout, role-based routing/middleware. |
| 2. Core Operations | Week 3–5 | Room CRUD, room-type management, guest management, walk-in booking, availability checking with exclusion constraints, check-in, check-out. |
| 3. Billing & Real-Time | Week 6–8 | Billing engine, invoice PDF generation, Socket.io integration for live room-status and notifications, housekeeping and restaurant modules. |
| 4. Reporting & Polish | Week 9–10 | Reports & analytics dashboards (Recharts), CSV/PDF export, staff management, audit log, feedback system. |
| 5. Deployment & Testing | Week 11–12 | CI/CD pipeline, deploy to Vercel + Railway/Render, end-to-end testing, performance tuning, documentation. |

## 8.2 Team Roles

| Role | Count | Responsibility |
|---|---|---|
| Project Lead | 1 | Architecture decisions, SRS ownership, sprint planning. |
| Frontend Developer | 1 | Next.js/React screens, Tailwind/shadcn styling, charts, forms. |
| Backend Developer | 1–2 | API routes, Prisma schema, business logic, WebSocket events. |
| QA / DevOps | 1 | Testing, CI/CD pipeline, deployment, environment/secret management. |

If built solo, one person covers all roles — prioritize Phases 1 through 3 first; Phases 4 and 5 can be scoped down for an MVP submission.

## 8.3 Testing Strategy

| Level | Focus |
|---|---|
| Unit Tests | Individual service functions, e.g. calculateBookingTotal(), checkRoomAvailability() (Jest/Vitest). |
| Integration Tests | API route → Prisma → PostgreSQL round trips — can a booking be created and correctly retrieved via the REST endpoint? |
| Real-Time Tests | Does a room-status change over the API trigger the expected WebSocket event on a connected client? |
| End-to-End Tests | Full user flows in a real browser (Playwright/Cypress): login → booking → check-in → checkout → invoice. |

Critical test scenarios: double-booking prevention when two bookings target the same room and dates; checkout with a zero balance; checkout with a partial payment; correct room-status transitions across the full lifecycle (Available → Reserved → Occupied → Cleaning → Available); account lockout after 5 failed logins; invoice-total accuracy (room + food + tax − discount); and WebSocket delivery to all connected roles within the 1-second NFR target.

## 8.4 Deployment Steps

- Provision a managed PostgreSQL instance on Railway/Render and run `prisma migrate deploy` to create the schema.
- Deploy the Express/NestJS backend to Railway/Render, with environment variables for DB URL, JWT secret, and OAuth client credentials set in the platform's secret manager.
- Deploy the Next.js frontend to Vercel, pointed at the backend's public API URL.
- Seed the initial Admin account and hotel_settings row during first deploy via a one-off migration/seed script.
- Enable the hosting provider's automated daily database backups.

## 8.5 Future Improvements (Phase 2 Roadmap)

- Customer-facing public booking website and native mobile app (React Native), reusing the same REST/WebSocket API.
- Integration with a real payment gateway (JazzCash, Easypaisa, or Stripe).
- OTA integration (Booking.com, Agoda) via a channel-manager API.
- Multi-property/chain management with centralized head-office reporting (multi-tenant schema).
- Loyalty/rewards program for repeat guests.
- AI-assisted dynamic pricing based on demand trends.
- Offline-first PWA caching for brief connectivity drops at the front desk.

# 9. Reference Implementation Structure

The following project layout is the recommended starting point for the Next.js + Node.js/Express + PostgreSQL build described in Section 4, split into a frontend app and a backend API as two deployable units (a monorepo with npm/pnpm workspaces is also a reasonable alternative).

## 9.1 Frontend Structure (Next.js)

```
hms-frontend/
├── package.json
├── next.config.js
├── tailwind.config.ts
├── app/
│   ├── (auth)/login/page.tsx
│   ├── (dashboard)/
│   │   ├── layout.tsx            # Sidebar + top nav shell
│   │   ├── dashboard/page.tsx
│   │   ├── rooms/page.tsx
│   │   ├── bookings/page.tsx
│   │   ├── checkin/page.tsx
│   │   ├── checkout/[bookingId]/page.tsx
│   │   ├── billing/page.tsx
│   │   ├── housekeeping/page.tsx
│   │   ├── restaurant/page.tsx
│   │   ├── staff/page.tsx
│   │   ├── reports/page.tsx
│   │   └── settings/page.tsx
├── components/
│   ├── ui/                       # shadcn/ui primitives
│   ├── room-status-grid.tsx
│   ├── data-table.tsx
│   └── charts/                   # Recharts wrappers
```

```
├── lib/
│   ├── api-client.ts            # Typed fetch/React Query hooks
│   ├── socket.ts                # Socket.io client singleton
│   └── auth.ts
└── tests/
```

## 9.2 Backend Structure (Node.js / Express + Prisma)

```
hms-backend/
├── package.json
├── prisma/
│   ├── schema.prisma            # Room, Booking, Guest, Invoice, etc.
│   └── seed.ts
├── src/
│   ├── server.ts                # Express app + Socket.io bootstrap
│   ├── config/
│   ├── middleware/
│   │   ├── auth.ts              # JWT verification, role guard
│   │   └── errorHandler.ts
│   ├── routes/
│   │   ├── auth.routes.ts
│   │   ├── rooms.routes.ts
│   │   ├── bookings.routes.ts
│   │   ├── billing.routes.ts
│   │   ├── housekeeping.routes.ts
│   │   ├── restaurant.routes.ts
│   │   └── reports.routes.ts
│   ├── services/
│   │   ├── auth.service.ts
│   │   ├── room.service.ts
│   │   ├── booking.service.ts
│   │   ├── billing.service.ts
│   │   ├── payment.service.ts
│   │   ├── housekeeping.service.ts
│   │   ├── food.service.ts
│   │   ├── report.service.ts
│   │   ├── notification.service.ts
│   │   └── audit.service.ts
│   ├── sockets/
│   │   └── index.ts             # WebSocket event emitters/handlers
│   └── utils/
│       └── invoicePdf.ts        # PDFKit/Puppeteer invoice renderer
└── tests/
    ├── booking.service.test.ts
    ├── billing.service.test.ts
    └── room.service.test.ts
```

## 9.3 Entry Point & Dependencies

```
// backend/package.json (key dependencies)
{
  "dependencies": {
    "express": "^4.19.2",
    "@prisma/client": "^5.16.0",
    "socket.io": "^4.7.5",
    "jsonwebtoken": "^9.0.2",
    "bcrypt": "^5.1.1",
    "passport": "^0.7.0",
    "passport-google-oauth20": "^2.0.0",
    "pdfkit": "^0.15.0",
    "zod": "^3.23.8"
  }
}

// backend/src/server.ts
import express from 'express';
import { createServer } from 'http';
import { Server } from 'socket.io';
```

```
import { authMiddleware } from './middleware/auth';
import roomRoutes from './routes/rooms.routes';
import bookingRoutes from './routes/bookings.routes';

const app = express();
const httpServer = createServer(app);
const io = new Server(httpServer, { cors: { origin: process.env.FRONTEND_URL } });

app.use(express.json());
app.use('/api/rooms', authMiddleware, roomRoutes);
app.use('/api/bookings', authMiddleware, bookingRoutes);

io.on('connection', (socket) => {
  socket.on('join:hotel', (hotelId) => socket.join(`hotel:${hotelId}`));
});

export { io };
httpServer.listen(process.env.PORT || 4000);
```

## 10. Summary

This revised blueprint replaces the desktop-first (Python/CustomTkinter/SQLite) architecture with a full-stack web architecture that matches the browser-based, multi-role dashboard UI already designed for the product: Next.js/TypeScript/Tailwind/shadcn on the frontend, Node.js/Express + Prisma on the backend, PostgreSQL as the database, and Socket.io for the real-time room-status and notification layer that the original desktop plan could only reach in a hypothetical Phase 2. All functional requirements, the database's entity relationships, and the UML/testing guidance from the original SRS carry over unchanged, since they describe hotel operations rather than implementation technology. The next steps are the same as before: transcribe Section 7 into the formal SRS template your course or client requires, render the ER/Use Case/Class/Activity/Sequence diagrams from Sections 6–7 in a diagramming tool, and begin implementation against the Phase 1 plan in Section 8.1.